ЛАБОРАТОРНАЯ РАБОТА №3
Gitleaks и pre-commit: защита от утечки секретов

Теория → пошаговая практика → проверка результата
2 академических часа · учебный проект vulnerable-app

Выполняем только на локальной учебной копии. Реальные секреты и чужие системы не используем.

1. Что вы изучите

Научиться находить секреты в файлах и истории Git, правильно реагировать и автоматически останавливать опасный коммит.

• Разберёте git и dir в Gitleaks.

• Удалите секрет из текущего кода.

• Настроите pre-commit.

• Проверите блокировку фиктивного токена.

• Объясните роль baseline и allowlist.

Опыт программирования не требуется: команды и фрагменты кода можно копировать. После каждого шага объясняется, что должно получиться и как проверить результат.

2. Теория простыми словами

Что такое секрет

Это пароль, токен, приватный ключ или строка подключения, которая даёт доступ. Наш ghp_-токен полностью фиктивный и нужен только для обучения. Никогда не используйте в лаборатории настоящие секреты.

Почему история Git важна

Если удалить токен новым коммитом, старый коммит останется доступен. При реальной утечке сначала отзывают/ротируют секрет, затем очищают код и историю по процедуре владельца и проверяют журналы использования.

Как работает Gitleaks

Правила ищут характерные строки и вычисляют энтропию — меру случайности. gitleaks dir сканирует файлы, gitleaks git — историю. --redact скрывает значение в выводе. Fingerprint идентифицирует находку.

Baseline и исключения

Baseline отделяет известные старые находки от новых. Allowlist и .gitleaksignore подавляют конкретные ложные срабатывания. Ни одно исключение не отзывает настоящий секрет.

Что такое pre-commit

Это менеджер локальных Git hooks. Он запускает проверки перед созданием коммита. Конфигурация .pre-commit-config.yaml хранится в репозитории. Локальный hook можно обойти, поэтому проверку повторяют в CI.

Зачем закреплять версию

rev определяет версию hook. Это делает проверку воспроизводимой. Обновления выполняют осознанно и проверяют, а не подставляют случайную ссылку.

3. Перед началом

• Распакуйте практикум и откройте терминал в каталоге vulnerable-app.

• Нужны Python 3.11+, браузер и терминал. Для ZAP и альтернативных запусков — Docker Desktop.

• Команды показаны для Git Bash. Для PowerShell используйте .venv\Scripts\Activate.ps1 вместо source.

• Создайте папку reports для результатов. Не публикуйте отчёты, пока не проверили их содержимое.

• Сохраняйте исходную копию проекта, чтобы сравнивать состояние до и после исправлений.

4. Практика по шагам

Шаг 1. Создать тренировочный Git-репозиторий

Зачем: Gitleaks git требует историю коммитов. Работаем только в копии vulnerable-app.

Что сделать:

1. Откройте терминал в vulnerable-app.

2. Инициализируйте Git, добавьте файлы и создайте первый коммит.

3. Если Git просит имя пользователя, задайте его локально для этого репозитория.

4. Убедитесь, что pre-commit hook пока не установлен.

Команда или код:

```
git init
git add .
git commit -m "Initial vulnerable training app"
# При необходимости:
git config user.name "Student Lab"
git config user.email "student@example.test"
git log --oneline
```

Ожидаемый результат: Команда git log --oneline показывает первый коммит с учебным токеном в config.py.

Шаг 2. Найти секрет двумя способами

Зачем: Нужно увидеть разницу между текущими файлами и историей.

Что сделать:

1. Запустите gitleaks dir для текущих файлов.

2. Запустите gitleaks git и сохраните JSON.

3. Выпишите RuleID, File, Line, Commit и Fingerprint. Сам секрет не копируйте.

4. Если бинарник не установлен, используйте Docker для dir.

Команда или код:

```
gitleaks dir --redact -v .
gitleaks git --redact -v --report-format json --report-path reports/gitleaks-before.json .

# Docker-вариант dir:
docker run --rm -v "${PWD}:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 dir --redact -v /src
```

Пример и объяснение: Пример: RuleID generic-api-key; File backend/config.py; Line 3; Secret [REDACTED].

Ожидаемый результат: Найден generic-api-key в backend/config.py, строка 3. Код выхода 1 означает обнаруженную утечку.

Шаг 3. Убрать секрет из текущего кода

Зачем: В реальном случае сначала отзывают секрет. Здесь токен учебный, поэтому отзыв только моделируем.

Что сделать:

1. Откройте backend/config.py.

2. Замените литерал токена на чтение переменной окружения.

3. Не добавляйте реальное значение в .env или другой файл репозитория.

4. Повторите dir и git: текущий файл чистый, история по-прежнему содержит находку.

Команда или код:

```
import os

DEMO_GITHUB_TOKEN = os.getenv("DEMO_GITHUB_TOKEN", "")
FLASK_SECRET_KEY = os.getenv("FLASK_SECRET_KEY", "dev-only-change-me")
```

Ожидаемый результат: gitleaks dir не находит новый литерал, а gitleaks git показывает старую запись. Это нормально и доказывает важность истории.

Шаг 4. Установить pre-commit

Зачем: Теперь настроим автоматическую проверку перед коммитом.

Что сделать:

1. Откройте .pre-commit-config.yaml и найдите repo, rev и id.

2. Установите пакет и Git hook.

3. Запустите проверку всех файлов.

4. После удаления текущего токена hook должен пройти; история при этом не переписана.

Команда или код:

```
python -m pip install pre-commit
pre-commit install
pre-commit run --all-files
```

Ожидаемый результат: pre-commit установлен в .git/hooks/pre-commit. При отсутствии новых секретов отображается Passed.

Шаг 5. Проверить блокировку опасного коммита

Зачем: Проверка нужна не для красивого отчёта, а для остановки ошибки.

Что сделать:

1. Создайте файл secret-demo.txt с другой фиктивной строкой ghp_ + 36 случайных букв и цифр.

2. Добавьте файл в index и попробуйте создать коммит.

3. Убедитесь, что Gitleaks остановил commit.

4. Удалите файл из index и с диска. Не отключайте hook.

Команда или код:

```
printf 'token=ghp_8ae31cacf141669ddfb5da7e31c8a2f3a4
' > secret-demo.txt
git add secret-demo.txt
git commit -m "Test secret blocking"
# Ожидается Failed. Затем:
```

```
git restore --staged secret-demo.txt
rm secret-demo.txt
```

Пример и объяснение: Если последовательность не обнаружена, используйте именно приведённую строку: некоторые простые последовательности вроде abcdef... входят в стандартный allowlist.

Ожидаемый результат: Коммит не создан, hook сообщает Finding, File и Line. После удаления файла повторная проверка проходит.

Шаг 6. Разобрать исключения и подготовить вывод

Зачем: Исключения должны уменьшать шум, а не скрывать утечки.

Что сделать:

1. Объясните, когда использовать baseline, а когда .gitleaksignore.

2. Предложите правило: новая подтверждённая находка блокирует CI.

3. Опишите порядок действий для настоящего секрета: владелец, отзыв, очистка, журналы, причина.

4. Укажите, почему отчёты нельзя публиковать без проверки содержимого.

Пример и объяснение: Baseline: известные старые находки не повторяются в списке новых. .gitleaksignore: точечный Fingerprint доказанного ложного срабатывания. Реальный токен не игнорируем.

Ожидаемый результат: В отчёте есть разница между текущим состоянием, историей, локальной проверкой и серверной политикой.

5. Что сдать в отчёте

• Команды и результаты dir/git.

• Карточка найденного секрета без его значения.

• Изменение config.py.

• Настройка pre-commit и доказательство Failed.

• Объяснение истории, baseline, исключений и CI.

В конце добавьте краткий вывод: какой дефект был найден, какая причина устранена, что проверено повторно и какие риски остались.

6. Проверь себя

1. Почему удаление токена не очищает историю?

2. Чем dir отличается от git?

3. Зачем нужен --redact?

4. Почему локального hook недостаточно?

5. Когда допустим allowlist?

7. Критерии оценивания

Критерий	Баллы
Git и сканирование	20
Устранение и история	20
Настройка hook	25

Проверка блокировки	20
Выводы и отчёт	15

Максимум — 100 баллов. За использование чужой системы без разрешения или публикацию настоящих секретов работа не принимается. Честно описанный отрицательный результат и ограничения инструмента являются нормальной частью анализа.

8. Источники

• ГОСТ Р 56939-2024: https://www.gostinfo.ru/catalog/Details/?id=8094263

• ГОСТ Р 71207-2024: https://protect.gost.ru/gost/details/9dd6f064-6014-4416-a3ca-ef72ce0a6c3a

• OWASP ASVS: https://owasp.org/www-project-application-security-verification-standard/

• Gitleaks: https://github.com/gitleaks/gitleaks

• pre-commit: https://pre-commit.com/